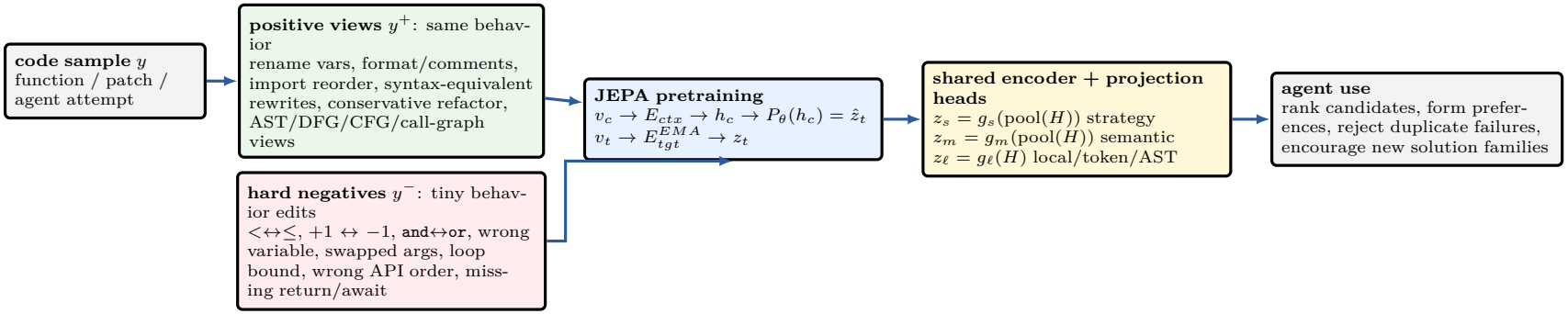
Code-JEPA for Agentic Code Self-Training

hard-negative-sensitive code geometry for reranking, hindsight feedback, and failure-memory exploration



1. Training objective

Normal JEPA learns predictive latent structure:

$$\mathcal{L}_{JEPA} = \|P_\theta(E_{ctx}(v_c)) - \text{sg}(E_{tgt}(v_t))\|_2^2.$$

For code, add invariance and hard-negative geometry:

$$\mathcal{L} = \mathcal{L}_{JEPA} + \lambda_p \mathcal{L}_{pos} + \lambda_n \mathcal{L}_{rank} + \lambda_\ell \mathcal{L}_{local}$$

$$\mathcal{L}_{rank} = \max(0,\, m + \text{sim}(z(y), z(y^-)) - \text{sim}(z(y), z(y^+))).$$

Key requirement: harmless rewrites collapse; one-character behavior changes can separate.

Positive transformations

- variable/function renaming when safe
- whitespace, comments, docstrings, quotes
- import reorder/split/merge when equivalent
- syntax-equivalent rewrites, conservative inlining/extraction
- alternate views: AST, DFG, CFG, call graph, dependency graph

Negative transformations

- off-by-one, wrong comparator, wrong boolean operator
- wrong variable, swapped args, wrong default/API order
- missing edge-case branch, return, exception, await
- mutate copy vs original, wrong sort direction

2. Multi-space similarity

Use one expensive transformer backbone, not separate full models. Projection heads define different geometries:

$$E(y) \to H, \quad z_s = g_s(H), \quad z_m = g_m(H), \quad z_\ell = g_\ell(H).$$

Pair type	strategy	semantic	local
format / rename / comments	+	+	low
behavior-preserving refactor	+	+	aligned
$<$ vs $\leq$, off-by-one	+	−	high span
wrong variable / swapped args	+	−	high span
same task, different algorithm	−	±	diffuse
unrelated code	−	−	high

+ = close, − = far, ± = close only if both solve the same spec.

Candidate interpretation

$\text{sim}_s(y_{new}, y_{bad})$	$\text{sim}_m(y_{new}, y_{bad})$	action
high	high	duplicate failure / rephrase: reject or deprioritize
high	low	same approach but meaningful fix: keep and verify
low	any	new solution family: keep for exploration

Local vectors prevent the bad failure mode where a global embedding treats `i < n` and `i <= n` as merely "almost identical".

3. Downstream self-training loops

A. Reference-known RLJF. Given prompt x, reference y_{ref}, samples $y_1 \dots y_k$:

$$score_i = \text{sim}_m(z_m(y_i), z_m(y_{ref})).$$

Use top/bottom samples as DPO/RLJF preference pairs; compare against CodeBERTScore/generic embeddings.

B. Agent eventually succeeds. Trajectory $y_1, \dots, y_T$ with verifier success at y_T:

$$y_{ref} := y_T.$$

Use hindsight: attempts closer to y_T become preferred over repeated failures; distill the successful trajectory and train away from earlier failure clusters.

C. Agent has no solution yet. All candidates fail. Store failure memory:

$$M_{bad} = \{C_j\}, \quad C_j = \text{clusters in } (z_s, z_m, z_\ell) \text{ space.}$$

For new candidate y:

$$S(y) = sanity + fit + \beta\, novelty_s - \gamma\, dup_{bad}.$$

Reject if it is close to a bad cluster in both strategy and semantic space with no high-impact local change. This rejects rephrasings of known-bad programs; it does *not* certify correctness.

Safety rule. Code-JEPA is a geometry/memory/ranking model. Positives require an anchor: reference, tests/verifier, accepted patch, or hindsight success. Pure novelty is exploration only.